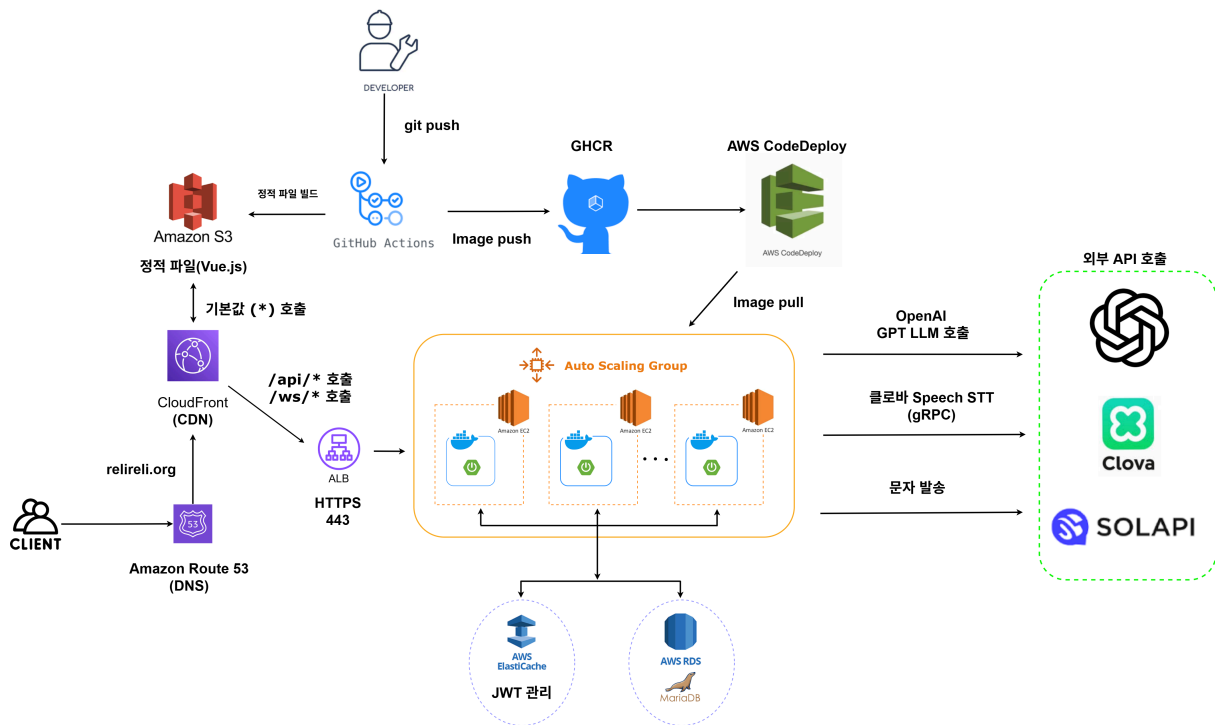


CI/CD 계획서

1. 시스템 아키텍처



2. CI/CD 인프라 구성

Relia 프로젝트는 GitHub Actions와 AWS 서비스를 연계하여 프론트엔드와 백엔드의 CI/CD 파이프라인을 구축하였다.

구분	기술
Source Control	GitHub
Continuous Integration (CI)	GitHub Actions
Container Registry	GitHub Container Registry (GHCR)
Deployment	Amazon S3, AWS CodeDeploy (Rolling Deployment), Amazon EC2 Auto Scaling
Infrastructure	Amazon EC2, Application Load Balancer (ALB), Amazon Route 53, Amazon CloudFront, AWS Identity and Access

구분	기술
	Management (IAM), AWS Systems Manager Parameter Store, AWS Certificate Manager (ACM)
Database & Cache	Amazon RDS (MariaDB), Amazon ElastiCache for Redis

3. 프론트엔드 CI/CD 파이프라인

3-1. Workflow 구조

배포 프로세스

1. `main` 브랜치에 Push
2. GitHub Actions Workflow 실행
3. Runner에서 Node.js 환경 구성 및 의존성 설치
4. Vite Build 수행(`dist` 생성)
5. Amazon S3에 정적 리소스 업로드
6. `index.html` 별도 업로드
7. Amazon CloudFront Cache Invalidation 수행
8. 사용자는 최신 정적 리소스 제공

배포 전략

- Amazon S3를 정적 웹 호스팅 저장소로 사용
 - Build 결과물을 Amazon S3에 업로드하여 정적 웹 페이지를 제공
- 최신 페이지 제공
 - `index.html` 은 `no-cache` 정책을 적용하여 항상 최신 버전 제공
- CloudFront 캐시 무효화
 - 배포 후 CloudFront 캐시를 제거하여 최신 리소스를 즉시 반영
- 변경되지 않는 정적 리소스는 브라우저 캐시를 활용하여 응답 속도 향상

관련 파일

- **.github/workflows/deploy.yaml**
<https://github.com/beyond-sw-camp/be25-fin-Linker-Relia-fe/blob/main/.github/workflows/deploy.yaml>

3-2. GitHub Variables / Secrets / Permissions

프론트엔드 배포에 필요한 설정 값은 GitHub Variables와 Secrets로 분리하여 관리하였다.

Variables에는 환경 설정 정보를 저장하고, Secrets에는 AWS IAM Role을 Assume하기 위한 인증 정보를 저장하였다.

- **GitHub Variables**

GitHub Variables	설명
AWS_REGION	배포를 수행할 AWS 리전
S3_BUCKET_NAME	Build 결과물을 업로드할 S3 버킷 이름
CLOUDFRONT_DISTRIBUTION_ID	CloudFront 캐시를 초기화할 대상 ID
VITE_API_BASE_URL	프론트엔드에서 사용할 백엔드 API 주소

- **GitHub Secrets**

Secret	설명
AWS_ROLE_TO_ASSUME	GitHub Actions가 OIDC 방식으로 AWS에 로그인하기 위해 사용할 IAM Role ARN

- **GitHub Permissions**

Permission	설명
contents: read	GitHub 저장소의 코드를 읽기 위한 권한
id-token: write	GitHub Actions가 OIDC를 통해 AWS IAM Role을 사용하기 위한 권한

3-3. Step별 동작 과정

Workflow는 다음과 같은 순서로 수행된다.

- **3-3-1. Checkout**

```
- name: Checkout
  uses: actions/checkout@v4
```

- GitHub 저장소의 최신 소스 코드를 Runner 환경으로 다운로드

- **3-3-2. Set up Node.js**

```
- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: 20
    cache: npm
```

- Node.js 20 실행 환경 구성
- npm Cache를 활용하여 의존성 설치 속도 향상

- **3-3-3. Install dependencies**

```
- name: Install dependencies
  run: npm ci
```

- `package-lock.json` 을 기준으로 프로젝트 의존성 설치
- 동일한 Build 환경 보장

- **3-3-4. Check build environment**

```
- name: Check build environment
  run: |
    test -n "${AWS_REGION}" || {
      echo "AWS_REGION repository variable is required."
    }
    exit 1
  }
  test -n "${S3_BUCKET_NAME}" || {
    echo "S3_BUCKET_NAME repository variable is required."
  }
  exit 1
```

```

    }
    test -n "${VITE_API_BASE_URL}" || {
        echo "VITE_API_BASE_URL repository variable is required." >&2
        exit 1
    }
}

```

- 필수 GitHub Variables 존재 여부 확인
- 환경 변수 누락 시 Build를 중단하여 잘못된 배포 방지

• 3-3-5. Build

```

- name: Build
  run: npm run build

```

- Vite Build를 수행하여 배포용 정적 리소스(`dist`) 생성

• 3-3-6. Configure AWS credentials

```

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-region: ${ env.AWS_REGION }
    role-to-assume: ${ secrets.AWS_ROLE_TO_ASSUME }

```

- GitHub OIDC를 이용하여 AWS 인증 수행
- Amazon S3 및 Amazon CloudFront 접근 권한 획득
- Access Key를 저장하지 않고 안전하게 AWS에 로그인

• 3-3-7. Upload static assets to S3

```

- name: Upload static assets to S3
  run: |
    aws s3 sync dist "s3://${S3_BUCKET_NAME}/" \
      --delete \
      --exclude "index.html" \

```

```
--cache-control "public,max-age=31536000,immutable" \
```

- JavaScript, CSS, 이미지 등 정적 리소스를 Amazon S3에 업로드
- `Immutable Cache` 를 적용하여 브라우저 캐시 성능 향상

• 3-3-8. Upload index.html

```
- name: Upload entry file to S3
  run: |
    aws s3 cp dist/index.html "s3://${S3_BUCKET_NAME}/index.html" \
      --cache-control "no-cache,no-store,must-revalidate" \
      --content-type "text/html; charset=utf-8"
```

- `index.html` 을 별도로 업로드
- `no-cache` 정책을 적용하여 항상 최신 진입 페이지 제공

• 3-3-9. Invalidate CloudFront

```
- name: Invalidate CloudFront
  if: ${ env.CLOUDFRONT_DISTRIBUTION_ID != '' }
  run: |
    aws cloudfront create-invalidation \
      --distribution-id "${CLOUDFRONT_DISTRIBUTION_ID}" \
      --paths "/*"
```

- Amazon CloudFront 캐시 무효화 수행
- 최신 Build 결과가 즉시 사용자에게 반영되도록 구성

4. 백엔드 CI/CD 파이프라인

4-1. Workflow 구조

배포 프로세스

1. 개발자 변경 사항을 `main` 브랜치에 Push
2. GitHub Actions 실행
3. `build` Job에서 Spring Boot 애플리케이션 Build
4. Docker 이미지 생성 후 GHCR에 Push
5. `deploy` Job에서 CodeDeploy Bundle 생성
6. Bundle을 S3에 업로드
7. CodeDeploy Deployment 생성
8. EC2에서 AppSpec 기준 Hook Script 실행
9. GHCR 이미지 Pull 후 Docker 컨테이너 교체
10. `/actuator/health` 로 정상 동작 검증

배포 전략

- **Rolling Deployment**
 - `CodeDeployDefault.HalfAtATime` 적용
 - 전체 인스턴스의 절반씩 순차 배포
- **무중단 배포 구성**
 - CodeDeploy와 ALB Target Group 연동
 - 배포 중인 인스턴스를 일시적으로 트래픽 대상에서 제외
 - Health Check 성공 후 다시 트래픽 대상에 포함
- **Health Check**
 - Spring Boot Actuator `/actuator/health` 사용
 - 정상 응답 확인 후 배포 성공 처리
- **Rollback**
 - 배포 실패 시 CodeDeploy 자동 Rollback 활성화
 - 이전 정상 버전으로 복구 가능

관련 파일

- `.github/workflows/deploy.yaml`

<https://github.com/beyond-sw-camp/be25-fin-Linker-Relia-be/blob/main/.github/workflows/deploy.yaml>

- appspec.yaml

<https://github.com/beyond-sw-camp/be25-fin-Linker-Relia-be/blob/main/appspec.yml>

- scripts/codedeploy/*

<https://github.com/beyond-sw-camp/be25-fin-Linker-Relia-be/tree/main/scripts/codedeploy>

4-2. GitHub Variables / Secrets / Permissions

백엔드 배포에 필요한 설정 값은 GitHub Variables와 Secrets로 분리하여 관리하였다.

Variables에는 배포에 필요한 설정 값과 SSM Parameter 이름을 저장하고, Secrets에는 AWS 인증을 위한 IAM Role 정보를 저장하였다.

- **GitHub Variables**

변수	설명
AWS_REGION	배포를 수행할 AWS 리전
GHCR_USERNAME_SSM_PARAMETER	GHCR 사용자명이 저장된 SSM Parameter 이름
GHCR_TOKEN_SSM_PARAMETER	GHCR Token이 저장된 SSM Parameter 이름
APP_ENV_SSM_PARAMETER	운영 환경 변수 값이 저장된 SSM Parameter 이름
CODEDEPLOY_S3_BUCKET	CodeDeploy Bundle을 업로드할 S3 버킷
CODEDEPLOY_S3_KEY_PREFIX	CodeDeploy Bundle이 저장될 S3 경로 Prefix
CODEDEPLOY_APPLICATION_NAME	CodeDeploy Application 이름
CODEDEPLOY_DEPLOYMENT_GROUP	CodeDeploy Deployment Group 이름

- **GitHub Secrets**

Secret	설명
AWS_ROLE_TO_ASSUME	GitHub Actions가 OIDC 방식으로 AWS에 로그인하기 위해 사용할 IAM Role ARN

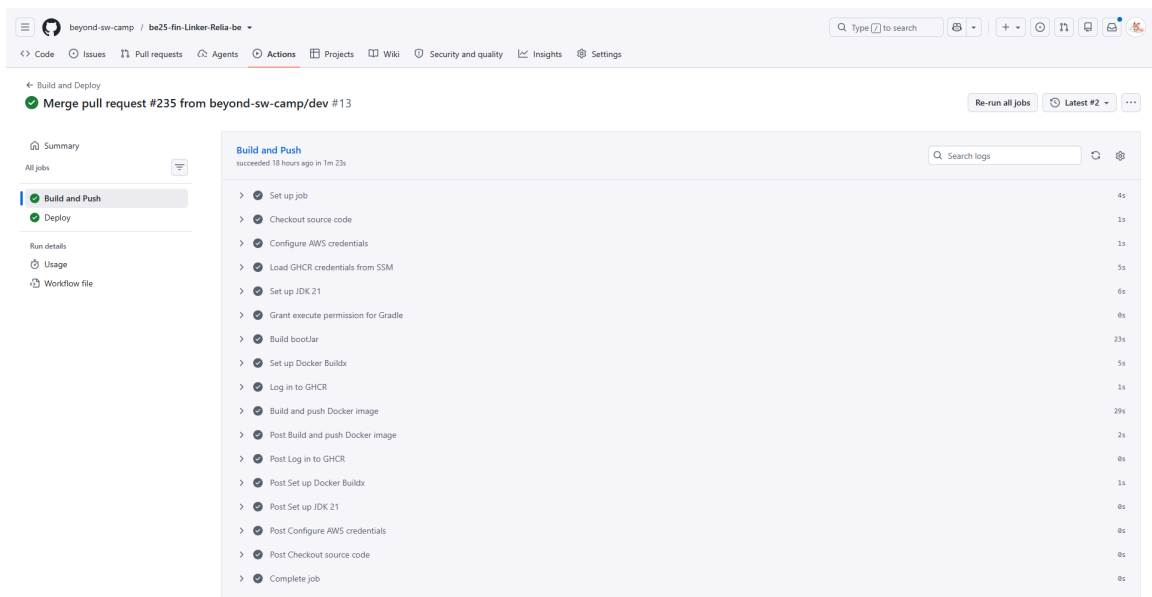
- **GitHub Permissions**

Permission	설명
contents: read	GitHub 저장소의 코드를 읽기 위한 권한
id-token: write	GitHub Actions가 OIDC를 통해 AWS IAM Role을 사용하기 위한 권한
packages: write	GHCR에 Docker 이미지를 Push하기 위한 권한

4-3. Build Job & Step별 동작 과정

Spring Boot 애플리케이션을 Build하고, Docker 이미지를 생성하여 GHCR에 Push하는 역할을 한다.

- **build Job 실행 결과**



- **Checkout source code**

```
- name: Checkout source code
  uses: actions/checkout@v4
```

- GitHub Actions Runner에 백엔드 소스 코드를 다운로드하여 이후 Build와 Docker 이미지 생성에 사용할 수 있도록 준비

- **Configure AWS credentials**

```
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-region: ${vars.AWS_REGION}
    role-to-assume: ${secrets.AWS_ROLE_TO_ASSUME}
```

- GitHub OIDC를 이용하여 AWS 인증 수행
- SSM Parameter Store 접근 권한 획득

• Load GHCR credentials from SSM

```
- name: Load GHCR credentials from SSM
  env:
    AWS_REGION: ${vars.AWS_REGION}
    GHCR_USERNAME_SSM_PARAMETER: ${vars.GHCR_USERNAME_SSM_PARAMETER}
    GHCR_TOKEN_SSM_PARAMETER: ${vars.GHCR_TOKEN_SSM_PARAMETER}
  run: |
    set -euo pipefail

    SHORT_SHA="${GITHUB_SHA::7}"

    GHCR_USERNAME="$(aws ssm get-parameter --region "${AWS_REGION}" --with-decryption --name "${GHCR_USERNAME_SSM_PARAMETER}" --query 'Parameter.Value' --output text)"
    GHCR_TOKEN="$(aws ssm get-parameter --region "${AWS_REGION}" --with-decryption --name "${GHCR_TOKEN_SSM_PARAMETER}" --query 'Parameter.Value' --output text)"

    {
      printf 'GHCR_USERNAME=%s\n' "${GHCR_USERNAME}"
      printf 'GHCR_TOKEN=%s\n' "${GHCR_TOKEN}"
      printf 'IMAGE_TAG=%s\n' "${SHORT_SHA}"
    } >> "${GITHUB_ENV}"
```

- GHCR 로그인에 필요한 사용자명과 Token을 SSM Parameter Store에서 조회

- 조회한 값을 이후 Step에서 사용할 수 있도록 GitHub Actions 환경 변수에 등록
- GHCR Token을 Workflow 파일에 직접 작성하지 않도록 구성

- **Set up JDK 21**

```
- name: Set up JDK 21
  uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: "21"
    cache: gradle
```

- Java 21 빌드 환경 구성
- Gradle Cache를 사용하여 빌드 속도 개선

- **Grant execute permission for Gradle**

```
- name: Grant execute permission for Gradle
  run: chmod +x gradlew
```

- Ubuntu Runner에서 gradlew 파일에 실행 권한을 부여

- **Build bootJar**

```
- name: Build bootJar
  run: ./gradlew clean bootJar --no-daemon
```

- Spring Boot 애플리케이션 Build 수행
- 실행 가능한 JAR 파일 생성

- **Set up Docker Buildx**

```
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3
```

- Docker 이미지 Build 환경 구성
- GitHub Actions Cache를 활용한 Docker Build 준비

- **Log in to GHCR**

```
- name: Log in to GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${ env.GHCR_USERNAME }
    password: ${ env.GHCR_TOKEN }
```

- GHCR에 Docker 이미지를 Push하기 위해 로그인

- **Build and push Docker image**

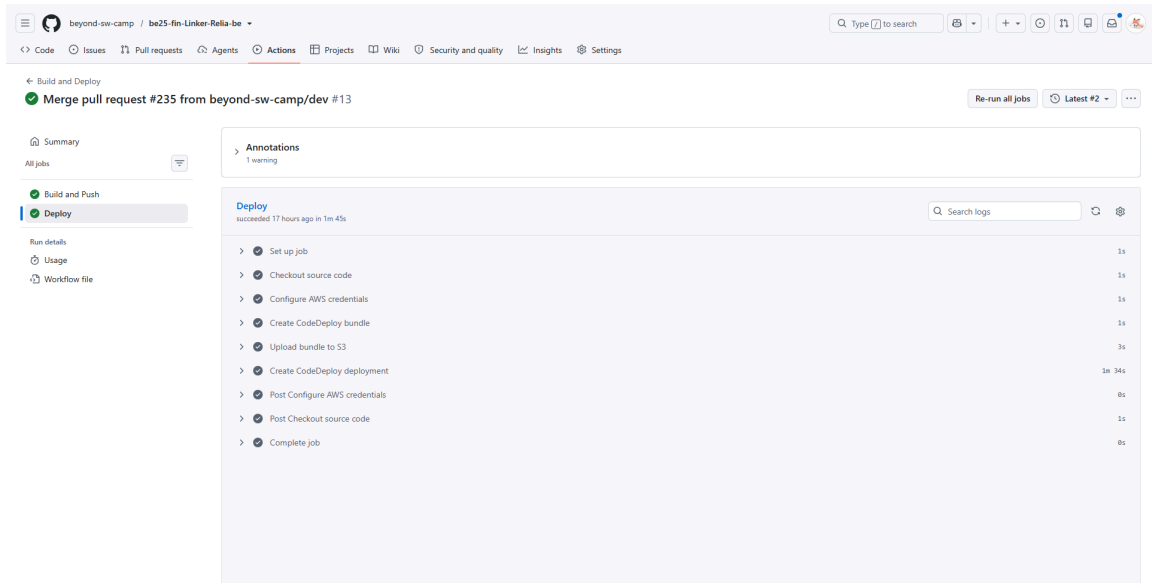
```
- name: Build and push Docker image
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
      ${ env.IMAGE_NAME }:${ env.IMAGE_TAG }
      ${ env.IMAGE_NAME }:latest
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

- Docker 이미지를 Build하고 GHCR에 Push
- `${ env.IMAGE_TAG }` 는 커밋 SHA 앞 7자리 Short SHA이다.
전체 SHA 대신 짧은 태그를 사용하여 이미지 버전을 관리하고, latest 태그도 함께 Push한다.
- `latest` 태그로 최신 이미지 식별
- Docker Layer Cache를 사용하여 빌드 속도 개선

4-4. Deploy Job & Step별 동작 과정

build Job이 성공한 뒤 실행된다. 생성된 Docker 이미지를 직접 EC2에 Push하는 것이 아니라, CodeDeploy Bundle을 생성하여 S3에 업로드하고 CodeDeploy 배포를 요청한다.

- **deploy Job 실행결과**



- **Checkout source code**

```
- name: Checkout source code
  uses: actions/checkout@v4
```

- Ubuntu Runner에 GitHub 저장소의 소스 코드를 다운로드하여 CodeDeploy Bundle 생성에 사용

- **Configure AWS credentials**

```
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-region: ${{ vars.AWS_REGION }}
    role-to-assume: ${{ secrets.AWS_ROLE_TO_ASSUME }}
```

- GitHub OIDC를 이용하여 AWS 인증 수행
- S3 업로드 및 CodeDeploy 배포 생성 권한 획득

- **Create CodeDeploy bundle**

```
- name: Create CodeDeploy bundle
  env:
    AWS_REGION: ${{ vars.AWS_REGION }}
    IMAGE_NAME: ${{ env.IMAGE_NAME }}
```

```

APP_ENV_SSM_PARAMETER: ${vars.APP_ENV_SSM_PARAMETER}
GHCR_USERNAME_SSM_PARAMETER: ${vars.GHCR_USERNAME_SSM_PARAMETER}
GHCR_TOKEN_SSM_PARAMETER: ${vars.GHCR_TOKEN_SSM_PARAMETER}
run: |
    set -euo pipefail

    SHORT_SHA="${GITHUB_SHA:0:7}"
    IMAGE_TAG="${SHORT_SHA}"
    REVISION_SUFFIX="${GITHUB_RUN_NUMBER}"

    BUNDLE_DIR="${RUNNER_TEMP}/codedeploy-bundle"
    ARCHIVE_PATH="${RUNNER_TEMP}/codedeploy-${GITHUB_SHA}-${REVISION_SUFFIX}.zip"

    rm -rf "${BUNDLE_DIR}" "${ARCHIVE_PATH}"
    mkdir -p "${BUNDLE_DIR}/deployment"

    rsync -a ./ "${BUNDLE_DIR}/" \
        --exclude ".git" \
        --exclude ".github" \
        --exclude ".gradle" \
        --exclude ".idea" \
        --exclude ".agents" \
        --exclude "build" \
        --exclude "deployment/nginx"

    {
        printf 'AWS_REGION=%s\n' "${AWS_REGION}"
        printf 'IMAGE_NAME=%s\n' "${IMAGE_NAME}"
        printf 'IMAGE_TAG=%s\n' "${IMAGE_TAG}"
        printf 'APP_ENV_SSM_PARAMETER=%s\n' "${APP_ENV_SSM_PARAMETER}"
        printf 'GHCR_USERNAME_SSM_PARAMETER=%s\n' "${GHCR_USERNAME_SSM_PARAMETER}"
        printf 'GHCR_TOKEN_SSM_PARAMETER=%s\n' "${GHCR_TOKEN_SSM_PARAMETER}"
    }

```

```
EN_SSM_PARAMETER}"
} > "${BUNDLE_DIR}/deployment/release.env"

cd "${BUNDLE_DIR}"
zip -qr "${ARCHIVE_PATH}" .
```

- CodeDeploy가 EC2에 전달할 배포 Bundle 생성
- `appspec.yml`, `docker-compose.prod.yaml`, Hook Script 등을 포함
- `deployment/release.env` 에 배포에 사용할 Docker 이미지 정보(이미지 이름, Short SHA 기반 태그)와 SSM Parameter 이름을 기록
- Bundle 파일명과 S3 업로드 경로에는 **Commit SHA + GitHub Actions Run Number**를 사용하여 배포 이력을 구분하고 동일 커밋 재배포 시 파일명 충돌을 방지
- 실제 Secret 값은 전달하지 않고 SSM Parameter 이름만 포함하여 민감 정보 노출을 방지

• Upload bundle to S3

```
- name: Upload bundle to S3
  env:
    AWS_REGION: ${vars.AWS_REGION}
    CODEDEPLOY_S3_BUCKET: ${vars.CODEDEPLOY_S3_BUCKET}
  }
  CODEDEPLOY_S3_KEY_PREFIX: ${vars.CODEDEPLOY_S3_KEY_PREFIX}
  run: |
    set -euo pipefail

    REVISION_SUFFIX="${GITHUB_RUN_NUMBER}"
    S3_KEY="${CODEDEPLOY_S3_KEY_PREFIX:-codedeploy}/releases/${GITHUB_SHA}-${REVISION_SUFFIX}.zip"
    aws s3 cp "${RUNNER_TEMP}/codedeploy-${GITHUB_SHA}-${REVISION_SUFFIX}.zip" "s3://${CODEDEPLOY_S3_BUCKET}/${S3_KEY}" --region "${AWS_REGION}"
    echo "S3_KEY=${S3_KEY}" >> "${GITHUB_ENV}"
```

- 생성한 CodeDeploy Bundle을 S3에 업로드

- Commit SHA와 GitHub Actions Run Number를 포함한 S3 Key를 사용하여 배포 이력을 구분하고 파일명 충돌을 방지
- 업로드한 S3 Key를 다음 Step에서 사용할 수 있도록 환경 변수에 저장

• Create CodeDeploy deployment

```
- name: Create CodeDeploy deployment
  env:
    AWS_REGION: ${vars.AWS_REGION}
    CODEDEPLOY_APPLICATION_NAME: ${vars.CODEDEPLOY_APPLICATION_NAME}
    CODEDEPLOY_DEPLOYMENT_GROUP: ${vars.CODEDEPLOY_DEPLOYMENT_GROUP}
    CODEDEPLOY_S3_BUCKET: ${vars.CODEDEPLOY_S3_BUCKET}
  run: |
    DEPLOYMENT_ID=$(aws deploy create-deployment \
      --region "${AWS_REGION}" \
      --application-name "${CODEDEPLOY_APPLICATION_NAME}" \
      --deployment-group-name "${CODEDEPLOY_DEPLOYMENT_GROUP}" \
      --description "GitHub Actions deployment ${GITHUB_SHA}" \
      --revision "revisionType=S3,s3Location={bucket=${CODEDEPLOY_S3_BUCKET},key=${S3_KEY},bundleType=zip}" \
      --query "deploymentId" \
      --output text)

    echo "Created deployment ${DEPLOYMENT_ID}"
    aws deploy wait deployment-successful --region "${AWS_REGION}" --deployment-id "${DEPLOYMENT_ID}"
```

- S3에 업로드한 Bundle을 기준으로 CodeDeploy 배포 생성
- CodeDeploy Application과 Deployment Group을 지정
- 배포가 성공할 때까지 GitHub Actions에서 대기
- 실패 시 GitHub Actions도 실패 처리

4-5. CodeDeploy AppSpec 구성

- CodeDeploy는 배포 Bundle에 포함된 `appspec.yml` 을 읽어 EC2에서 수행할 배포 절차를 결정한다.

The screenshot shows the AWS CodeDeploy console for a deployment named 'reli-a' on an EC2 instance. The console displays the following information:

- 배포 세부 정보 (Deployment Details):**
 - 애플리케이션 (Application): `reli-a`
 - 배포 ID (Deployment ID): `d-EIEH84A8J`
 - 상태 (Status): `성공 (Success)`
 - 배포 구성 (Deployment Configuration): `CodeDeployDefault.LinuxOnPrem`
 - 배포 그룹 (Deployment Group): `reli-a-deploy`
 - 시작 (Start): `Auto Scaling 그룹 작업`
- 배포 설명 (Deployment Description):**

Instance i-0ec95e30fd692c76e was launched by Auto Scaling group reli-a-auto-scaling hook CodeDeploy-managed-automatic-launch-deployment-hook-reli-a-deploy-9a7c229a-87f9-4056-96a9-c5a8e9000c50
- 개정 세부 정보 (Revision Details):**
 - 개정 위치 (Revision Location): `s3://reli-a-code-deploy/code-deploy/prod/releases/06710a2f1c83a143a94a81221bb0969071e5d7ec.zip`
 - 개정 생성 (Revision Created): `18시간 전`
 - 개정 설명 (Revision Description): `Application revision registered by Deployment ID: d-ZBXYXNW7J`
- 이벤트 로그 (Event Log):**

이벤트 (Event)	기간 (Duration)	상태 (Status)	오류 코드 (Error Code)	시작 시간 (Start Time)	종료 시간 (End Time)
ApplicationStop	1초 미만	성공 (Success)	-	6월 27, 2026 3:10 오후 (UTC+9:00)	6월 27, 2026 3:10 오후 (UTC+9:00)
DownloadBundle	1초 미만	성공 (Success)	-	6월 27, 2026 3:10 오후 (UTC+9:00)	6월 27, 2026 3:10 오후 (UTC+9:00)
BeforeInstall	1초 미만	성공 (Success)	-	6월 27, 2026 3:10 오후 (UTC+9:00)	6월 27, 2026 3:10 오후 (UTC+9:00)
Install	1초 미만	성공 (Success)	-	6월 27, 2026 3:10 오후 (UTC+9:00)	6월 27, 2026 3:10 오후 (UTC+9:00)
AfterInstall	1초	성공 (Success)	-	6월 27, 2026 3:10 오후 (UTC+9:00)	6월 27, 2026 3:10 오후 (UTC+9:00)
ApplicationStart	19초	성공 (Success)	-	6월 27, 2026 3:10 오후 (UTC+9:00)	6월 27, 2026 3:10 오후 (UTC+9:00)
ValidateService	1분 52초	성공 (Success)	-	6월 27, 2026 3:11 오후 (UTC+9:00)	6월 27, 2026 3:15 오후 (UTC+9:00)
BeforeAllowTraffic	1초 미만	성공 (Success)	-	6월 27, 2026 3:13 오후 (UTC+9:00)	6월 27, 2026 3:13 오후 (UTC+9:00)
AllowTraffic	15초	성공 (Success)	-	6월 27, 2026 3:13 오후 (UTC+9:00)	6월 27, 2026 3:13 오후 (UTC+9:00)
AfterAllowTraffic	1초 미만	성공 (Success)	-	6월 27, 2026 3:13 오후 (UTC+9:00)	6월 27, 2026 3:13 오후 (UTC+9:00)

- `appspec.yml` 에는 배포 파일의 복사 위치와 배포 생명주기(Lifecycle)별로 실행할 Hook Script가 정의되어 있다.

```
version: 0.0
os: linux

files:
- source: /
  destination: /home/ubuntu/app
  overwrite: true

permissions:
- object: /home/ubuntu/app
  pattern: "*"
  owner: ubuntu
  group: ubuntu
```

```

hooks:
  ApplicationStop:
    - location: scripts/codedeploy/application_stop.sh
      timeout: 300
      runas: root
  AfterInstall:
    - location: scripts/codedeploy/after_install.sh
      timeout: 300
      runas: root
  ApplicationStart:
    - location: scripts/codedeploy/application_start.sh
      timeout: 600
      runas: root
  ValidateService:
    - location: scripts/codedeploy/validate_service.sh
      timeout: 600
      runas: root

```

4-6. CodeDeploy Hook Script 구성

`appspec.yml`에 정의된 Hook Script는 EC2 내부에서 실제 배포 작업을 수행한다.

- 실제 배포 순서는 다음과 같다.

```
ApplicationStop --> AfterInstall --> ApplicationStart --> ValidateService
```

- **application_stop.sh**

```

#!/usr/bin/env bash
set -euo pipefail

APP_DIR="/home/ubuntu/app"

if [ ! -f "${APP_DIR}/docker-compose.prod.yaml" ]; then
  exit 0
fi

```

```
cd "${APP_DIR}"
docker compose -f docker-compose.prod.yaml down --remove
-orphans || true
```

- 기존에 실행 중이던 Docker Compose 기반 애플리케이션 컨테이너 종료
- `docker-compose.prod.yaml` 파일이 없는 경우 정상 종료
- 기존 Compose 파일에 정의되지 않은 orphan 컨테이너도 함께 정리

- **after_install.sh**

```
#!/usr/bin/env bash
set -euo pipefail

APP_DIR="/home/ubuntu/app"

mkdir -p "${APP_DIR}"
cd "${APP_DIR}"

chmod +x scripts/codedeploy/*.sh

if [ ! -f deployment/release.env ]; then
    echo "deployment/release.env file is required for prod
    uction deployment." >&2
    exit 1
fi

set -a
source deployment/release.env
set +a

if [ -z "${APP_ENV_SSM_PARAMETER:-}" ]; then
    echo "APP_ENV_SSM_PARAMETER is required." >&2
    exit 1
fi

if [ -z "${GHCR_USERNAME_SSM_PARAMETER:-}" ]; then
    echo "GHCR_USERNAME_SSM_PARAMETER is required." >&2
    exit 1
fi
```

```

fi

if [ -z "${GHCR_TOKEN_SSM_PARAMETER:-}" ]; then
    echo "GHCR_TOKEN_SSM_PARAMETER is required." >&2
    exit 1
fi

TMP_JSON=$(mktemp)
aws ssm get-parameter \
    --region "${AWS_REGION}" \
    --with-decryption \
    --name "${APP_ENV_SSM_PARAMETER}" \
    --output json > "${TMP_JSON}"

python3 - "${TMP_JSON}" "${APP_DIR}/.env" <<'PY'
import json
import pathlib
import sys

payload_path = pathlib.Path(sys.argv[1])
target_path = pathlib.Path(sys.argv[2])
data = json.loads(payload_path.read_text(encoding="utf-8"))
target_path.write_text(data["Parameter"]["Value"] +
    "\n", encoding="utf-8")
PY

rm -f "${TMP_JSON}"

```

- 배포 파일 설치 후 실행 준비 수행
- Hook Script 실행 권한 부여
- `deployment/release.env` 를 읽어 이번 배포 설정 로드
- SSM Parameter Store에서 운영 환경 변수 조회
- 조회한 운영 환경 변수 값을 EC2 내부 `.env` 파일로 생성
- DB, Redis, JWT Secret 등 민감 정보가 GitHub 저장소에 포함되지 않도록 구성

- **application_start.sh**

```
#!/usr/bin/env bash
set -euo pipefail

APP_DIR="/home/ubuntu/app"

cd "${APP_DIR}"

set -a
source deployment/release.env
set +a

GHCR_USERNAME="$(aws ssm get-parameter --region "${AWS_REGION}" --with-decryption --name "${GHCR_USERNAME_SSM_PARAMETER}" --query 'Parameter.Value' --output text)"
GHCR_TOKEN="$(aws ssm get-parameter --region "${AWS_REGION}" --with-decryption --name "${GHCR_TOKEN_SSM_PARAMETER}" --query 'Parameter.Value' --output text)"

printf '%s' "${GHCR_TOKEN}" | docker login ghcr.io -u
"${GHCR_USERNAME}" --password-stdin

docker compose -f docker-compose.prod.yaml pull app
docker compose -f docker-compose.prod.yaml up -d app
```

- SSM Parameter Store에서 GHCR 로그인 정보 조회
- GHCR 로그인 수행
- `docker-compose.prod.yaml` 기준 최신 Docker 이미지 Pull
- `app` 컨테이너 백그라운드 실행

- **validate_service.sh**

```
#!/usr/bin/env bash
set -euo pipefail

APP_DIR="/home/ubuntu/app"
```

```

cd "${APP_DIR}"

set -a
source .env
set +a

for attempt in $(seq 1 30); do
    if curl -fsS "http://127.0.0.1:${APP_PORT:-8080}/actuator/health" | grep -q '"status":"UP"'; then
        exit 0
    fi
    sleep 10
done

docker compose -f docker-compose.prod.yaml logs --tail=200 app
exit 1

```

- 새로 실행된 Spring Boot 애플리케이션 정상 기동 여부 검증
- `/actuator/health` 응답의 `"status":"UP"` 확인
- 최대 30회, 10초 간격으로 재시도
- 실패 시 컨테이너 로그를 출력하고 배포 실패 처리
- CodeDeploy 자동 Rollback 설정과 연계하여 이전 정상 버전으로 복구 가능